

Desarrollo con DDD y Vue

<https://gemini.google.com/app/40b108414d8997e0>

User prompt: Dime como hacer esto, usando DDD, Vue y JavaScript

Response: Para desarrollar la Práctica Calificada cumpliendo con el puntaje máximo de la rúbrica (20/20) y las restricciones de **Domain-Driven Design (DDD)**, arquitectura por capas, PrimeVue y Vue 3 con Composition API, sigue este procedimiento paso a paso.

1. Creación del Proyecto y Dependencias

Ejecuta en tu terminal los siguientes comandos para inicializar el proyecto con Vite y JavaScript:

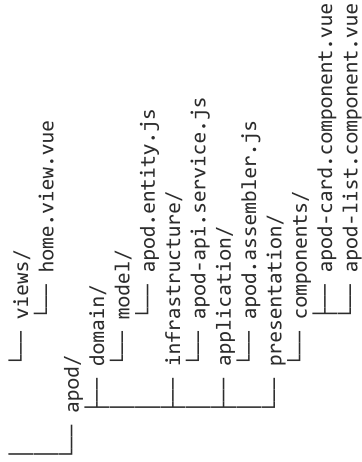
```
# Reemplaza con tu NRC y código (ejemplo: pc17432u202019287)
npm create vite@latest pc17432u202019287 -- --template vue
cd pc17432u202019287

# Instalación de librerías requeridas
npm install axios vue-i18n@9 primevue@^3 primeicons primeflex
npm install
```

2. Estructura de Directorios (DDD + Arquitectura por Capas)

Organiza la carpeta `src` en los dos subdominios solicitados: `shared` y `apod`, separando responsabilidades en capas (`domain`, `infrastructure`, `application`, `presentation`):

```
src/
├── assets/
│   └── main.css
├── i18n.js
├── main.js
├── App.vue
├── shared/
│   └── presentation/
│       └── components/
│           ├── the-header.component.vue
│           └── the-footer.component.vue
```



3. Configuración de Internacionalización y PrimeVue

Configuración de i18n (src/i18n.js)

Crea el archivo de internacionalización con soporte para inglés (por defecto) y español:

```

/**
 * @summary Internationalization configuration for English and Spanish
 * @author Juan Sebastian Estupiñan Olortegui
 */
import { createI18n } from 'vue-i18n';

const messages = {
  en: {
    brand: 'NASA',
    title: 'Astronomy Pictures of the Day',
    seeOnNasa: 'See on NASA',
    fallbackMedia: 'Media not available',
    footer: {
      copyright: 'Copyright ©2025 The NASA, inc All rights reserved',
      developedBy: 'Developed by'
    }
  },
  es: {

```

```

    brand: 'NASA',
    title: 'Imágenes Astronómicas del Día',
    seeOnNasa: 'Ver en NASA',
    fallbackMedia: 'Multimedia no disponible',
    footer: {
      copyright: 'Copyright ©2025 The NASA, inc All rights reserved',
      developedBy: 'Desarrollado por'
    }
  }
};

const i18n = createI18n({
  legacy: false,
  locale: 'en',
  fallbackLocale: 'en',
  messages
});

export default i18n;

```

Punto de Entrada (src/main.js)

Configura PrimeVue, el tema Material (md-light-indigo), PrimeFlex y registra los componentes con el prefijo obligatorio pv-:

```

/**
 * @summary Application bootstrap file configuring PrimeVue, i18n,
 * @author Juan Sebastian Estupiñan Olortegui
 */
import { createApp } from 'vue';
import App from './App.vue';
import i18n from './i18n';

import PrimeVue from 'primevue/config';
import Toolbar from 'primevue/toolbar';
import Card from 'primevue/card';
import Button from 'primevue/button';
import SelectButton from 'primevue/selectbutton';

import 'primevue/resources/themes/md-light-indigo/theme.css';

```

```
import 'primevue/resources/primevue.min.css';
import 'primeicons/primeicons.css';
import 'primeflex/primeflex.css';
import './assets/main.css';

const app = createApp(App);

app.use(PrimeVue, { ripple: true });
app.use(i18n);

// Registro global con prefijo pv-
app.component('pv-toolbar', Toolbar);
app.component('pv-card', Card);
app.component('pv-button', Button);
app.component('pv-select-button', SelectButton);

app.mount('#app');
```

4. Subdominio: apod

Modelo de Dominio (src/apod/domain/model/apod.entity.js)

Nótese que el nombre del archivo físico es apod.entity.js, pero la clase lógica es Apod:

```
/**
 * @summary Domain entity representing an Astronomy Picture of the
 * @author Juan Sebastian Estupiñan Olortegui
 */
export class Apod {
  /**
   * @param {Object} params
   * @param {string} params.title
   * @param {string} params.date
   * @param {string} params.explanation
   * @param {string} params.url
   * @param {string} params.mediaType
   */
  constructor({ title, date, explanation, url, mediaType }) {
    this.title = title || '';
  }
}
```

```

    this.date = date || '';
    this.explanation = explanation || '';
    this.url = url || '';
    this.mediaType = mediaType || 'image';
  }

  /**
   * Converts the title to kebab-case for the official NASA APOD l
   * @returns {string}
   */
  get kebabTitle() {
    return this.title
      .replace(/([a-z])([A-Z])/g, '$1-$2')
      .replace(/[\s_]+/g, '-')
      .toLowerCase()
      .replace(/^[^a-z0-9-]/g, '');
  }

  /**
   * Generates the external NASA APOD URL.
   * @returns {string}
   */
  get nasaUrl() {
    return `https://apod.nasa.gov/apod/${this.kebabTitle}`;
  }
}

```

Servicio de Infraestructura (src/apod/infrastructure/apod-api.service.js)

```

/**
 * @summary Infrastructure service handling HTTP calls to the NASA/
 * @author Juan Sebastian Estupiñan Olortegui
 */
import axios from 'axios';

const NASA_API_BASE_URL = 'https://api.nasa.gov/planetary/apod';
const DEMO_KEY = 'DEMO_KEY';

export class ApodApiService {

```

```

/**
 * Fetches a list of astronomy pictures.
 * @param {number} count
 * @returns {Promise<Array>}
 */
async getAstronomyPictures(count = 10) {
  const response = await axios.get(`${NASA_API_BASE_URL}?api_key=
  return response.data;
}
}

```

Ensamblador (src/apod/application/apod.assembler.js)

Aplica el patrón Assembler para transformar los recursos recibidos de la API en entidades del dominio:

```

/**
 * @summary Assembler to transform NASA API raw response resource€
 * @author Juan Sebastian Estupiñan Olortegui
 */
import { Apod } from '../domain/model/apod.entity.js';

export class ApodAssembler {
  /**
   * Transforms a single API resource into an Apod entity.
   * @param {Object} resource
   * @returns {Apod}
   */
  static toEntityFromResource(resource) {
    return new Apod({
      title: resource.title,
      date: resource.date,
      explanation: resource.explanation,
      url: resource.url,
      mediaType: resource.media_type
    });
  }
}

/**
 * Transforms an array of API resources into an array of Apod er

```

```

* @param {Array} resources
* @returns {Array<Apod>}
*/
static toEntitiesFromResources(resources) {
  if (!Array.isArray(resources)) return [];
  return resources.map((resource) => this.toEntityFromResource(r
}
}

```

Componente Card (src/apod/presentation/components/apod-card.component.vue)

```

<!--
  @summary Card component rendering a single APOD item with ARIA
  @author Juan Sebastian Estupiñan Olortegui
-->
<script setup>
import { computed } from 'vue';
import { useI18n } from 'vue-i18n';
import { Apod } from '../domain/model/apod.entity.js';

const { t } = useI18n();

const props = defineProps({
  apod: {
    type: Apod,
    required: true
  }
});

const isVideo = computed(() => props.apod.mediaType === 'video');
const hasMedia = computed(() => Boolean(props.apod.url));
</script>

<template>
  <pv-card class="h-full flex flex-column justify-content-between
  <template #header>
    <div v-if="hasMedia" class="w-full" style="height: 220px; o
      <iframe
        v-if="isVideo"

```

```

:src="apod.url"
class="w-full h-full border-none"
title="NASA APOD video"
allowfullscreen
</iframe>

</div>
<div v-else class="w-full flex align-items-center justify-center"
  <span class="text-gray-600">{{ t('fallbackMedia') }}</span>
</div>
</template>

<template #title>
<h3 class="text-lg font-bold m-0 line-clamp-1" :title="apod.
</template>

<template #subtitle>
<span class="text-sm text-gray-500">{{ apod.date }}</span>
</template>

<template #content>
<p class="text-sm line-clamp-3 text-700 m-0" :title="apod.e
  {{ apod.explanation }}
</p>
</template>

<template #footer>
<div class="flex justify-content-end">
  <a href="apod.nasaurl" target="_blank" rel="noopener none
    <pv-button
      :label="t('seeOnNasa')"
      icon="pi pi-external-link"
      class="p-button-sm p-button-outlined"

```

```

      :aria-label="`${t('seeOnNasa')}`: ${apod.title}`"
    />
  </a>
</div>
</template>
</pv-card>
</template>

<style scoped>
.line-clamp-1 {
  display: -webkit-box;
  -webkit-line-clamp: 1;
  -webkit-box-orient: vertical;
  overflow: hidden;
}
.line-clamp-3 {
  display: -webkit-box;
  -webkit-line-clamp: 3;
  -webkit-box-orient: vertical;
  overflow: hidden;
}
</style>

```

Componente Lista (src/apod/presentation/components/apod-list.component.vue)

```

<!--
@summary Grid list rendering multiple APOD cards using PrimeFle
@author Juan Sebastian Estupiñan Olortegui
-->
<script setup>
import ApodCard from './apod-card.component.vue';

defineProps({
  apods: {
    type: Array,
    required: true
  }
});
</script>

```

```

<template>
  <section class="grid p-fluid" aria-label="Astronomy pictures li:
    <div
      v-for="(apod, index) in apods"
      :key="`${apod.title}-${index}`"
      class="col-12 sm:col-6 md:col-4 lg:col-3 p-2"
    >
      <apod-card :apod="apod" />
    </div>
  </section>
</template>

```

5. Subdominio: shared

Header con Logo Clearbit y Selector de Idioma

(src/shared/presentation/components/the-header.component.vue)

```

<!--
  @summary Application header with Clearbit NASA logo and language
  @author Juan Sebastian Estupiñan Olortegui
-->
<script setup>
  import { ref, watch } from 'vue';
  import { useI18n } from 'vue-i18n';

  const { locale, t } = useI18n();

  const languageOptions = [
    { label: 'EN', value: 'en' },
    { label: 'ES', value: 'es' }
  ];

  const selectedLanguage = ref(locale.value);

  watch(selectedLanguage, (newLang) => {
    if (newLang) {
      locale.value = newLang;
    }
  })

```

```

});
</script>

<template>
  <header role="banner">
    <pv-toolbar class="border-noround surface-card px-4 py-2 shadow">
      <template #start>
        <div class="flex align-items-center gap-3">
          
          <h1 class="text-xl font-bold m-0">{{ t('brand') }}</h1>
        </div>
      </template>

      <template #end>
        <div class="flex align-items-center">
          <pv-select-button
            v-model="selectedLanguage"
            :options="languageOptions"
            optionLabel="label"
            optionValue="value"
            :allowEmpty="false"
            aria-label="Language selector"
          />
        </div>
      </template>
    </pv-toolbar>
  </header>
</template>

```

Footer (src/shared/presentation/components/the-footer.component.vue)

```

<!--
  @summary Application footer presenting NASA copyright and developer
  @author Juan Sebastian Estupian Olortegui

```

```

-->
<script setup>
import { useI18n } from 'vue-i18n';

const { t } = useI18n();
</script>

<template>
  <footer class="surface-900 text-0 text-center py-4 mt-auto" role
    <p class="m-0 text-sm mb-1">{{ t('footer.copyright') }}</p>
    <p class="m-0 text-xs text-400">
      {{ t('footer.developedBy') }} u202019287 - Juan Sebastian Es
    </p>
    </footer>
  </template>

```

Vista Home (src/shared/presentation/views/home.view.vue)

```

<!--
  @summary Main home view orchestrating the APOD API call and rendering
  @author Juan Sebastian Estupiñan Olortegui
-->
<script setup>
import { ref, onMounted } from 'vue';
import { useI18n } from 'vue-i18n';
import TheHeader from '../components/the-header.component.vue';
import TheFooter from '../components/the-footer.component.vue';
import ApodList from '../..//apod/presentation/components/apod-list';
import { ApodApiService } from '../..//apod/infrastructure/apod-api-service';
import { ApodAssembler } from '../..//apod/application/apod-assembler';

const { t } = useI18n();
const apods = ref([]);
const loading = ref(true);

const apodApiService = new ApodApiService();

const fetchApods = async () => {
  try {
    loading.value = true;

```

```

const rawData = await apodApiService.getAstronomyPictures(10);
apods.value = ApodAssembler.toEntitiesFromResources(rawData);
} catch (error) {
  console.error('Error fetching APODs:', error);
} finally {
  loading.value = false;
}
});

onMounted(() => {
  fetchApods();
});
</script>

<template>
  <div class="min-h-screen flex flex-column surface-ground">
    <header />
    <main class="flex-grow-1 p-4 max-w-7xl mx-auto w-full" role="main">
      <h2 class="text-2xl font-bold mb-4 text-800">{{ t('title') }}
      <div v-if="loading" class="text-center py-6">
        <i class="pi pi-spin pi-spinner text-4xl text-primary" aria
      </div>
    </main>
    <apod-list v-else :apods="apods" />
  </div>
  <the-footer />
</div>
</template>

```

6. Archivo Raiz (src/App.vue)

Dado que el enunciado marca el enrutador (*routing*) fuera del alcance, App.vue renderiza directamente la vista principal:

```

<!--
  @summary Root component displaying the main application view.
  @author Juan Sebastian Estupiñan Olortegui
-->
<script setup>
import HomeView from './shared/presentation/views/home.vue';
</script>

```

```
<template>
  <home-view />
</template>
```

7. Documentación README.md

Crea el archivo README.md en la raíz del proyecto redactado en inglés como exige la rúbrica:

```
# NASA Astronomy Pictures Web Application

## Description
A domain-driven web application built with Vue 3, Composition API,

## Features
- **Toolbar**: Institution branding with Clearbit Logo API and bil
- **APOD Grid**: Responsive card presentation displaying title, da
- **DDD Architecture**: Domain, Infrastructure, Application, and F
- **Accessibility**: Native semantic HTML5 elements combined with

## Dependencies
- Vue 3 (Composition API)
- Vite
- PrimeVue & PrimeFlex
- Axios
- Vue-i18n

## Author
- **Student Code**: u202019287
- **Name**: Juan Sebastian Estupiñan Olortegui
```

8. Verificaciones Finales Antes del Empaquetado

- 1. Compilación y Ejecución:** Ejecuta `npm run build` y luego `npm run preview` (o `npm run dev`) para verificar que no existan errores de compilación ni advertencias en la consola.
- 2. Eliminación de node_modules:** Antes de empaquetar, elimina la carpeta `node_modules` y `.git` para que el archivo no sea excesivamente pesado.

3. **Empaquetado:** Comprime la carpeta del proyecto en formato .zip con el nombre exacto indicado en las instrucciones:

```
pc1[nrc][ucode].zip  
# Ejemplo: pc17432u202019287.zip
```
